1. Implement apply_operation()

```
def apply_operation(numbers, operation):
return list(map(operation, numbers))

numbers = [1, 2, 3, 4, 5]
result = apply_operation(numbers, lambda x: x ** 2)
print(result)
```

2. Implement filter_items()

```
def filter_items(items, predicate):
return list(filter(predicate, items))

items = [1, 2, 3, 4, 5, 6, 7, 8]
result = filter_items(items, lambda x: x % 2 == 0)
print(result)
```

3. Implement compose()

```
def compose(f, g):
return lambda x: f(g(x))

g = lambda x: x * 2
f = lambda x: x + 10

result = compose(f, g)
print(result(5))
```

4. Implement sort_by_key()

```
def sort_by_key(objects, key_func):
return sorted(objects, key=key_func)

students = [
{"name": "Sathish", "age": 22},
{"name": "Rajesh", "age": 20},
{"name": "Anil", "age": 24}
]
```

```
result = sort_by_key(students, lambda x: x["age"])
print(result)
```

5. Convert strings to integer squares using map and lambda

```
li = ["1", "2", "3", "4", "5"]
result = list(map(lambda x: int(x) ** 2, li))
print(result)
```

6. Implement cumulative_product() using reduce

```
from functools import reduce

def cumulative_product(numbers):
return reduce(lambda x, y: x * y, numbers, 1)

numbers = [1, 2, 3, 4, 5, 6, 7, 8]
result = cumulative_product(numbers)
print(result)
```